Department of Computer Science and Engineering

CSE 1325 Project Documentation

CUBICLE

Students' names, surnames, IDs:
1. Andy Ta, 1002256916
2. Duc Minh Nguyen, 1002289551
3. Andrew Butts, 1002258140
4.

Mentor: Marika Apostolova

PROJECT LOGO

Object oriented programming CSE 1325

Student declaration:

We declare that:

- *We understand what is meant by plagiarism*
- *The implication of plagiarism has been explained to me by our professor*
- *This assignment is all team own work and we have acknowledged any use of the published and unpublished works of other people.*

1 Student name, surname, ID and **signature:** **Andy Ta 1002256916**
2 Student name, surname, ID and **signature:** **Duc Minh Nguyen 1002289551**
3 Student name, surname, ID and **signature:** **Andrew Butts 1002258140**
4 Student name, surname, ID and **signature:** …….....

Date: **4/18/2026**

	Total number of pages including this cover page	## 15

Class Code / Group	CSE 1325
Lecturer's Name	MARIKA APOSTOLOVA

ALL ANSWERS AT THE END!

1. **Project Introduction**
   Describe briefly the main idea of the project.

2. **Project Description**
   A brief summary of the goals, the objectives, and the need for the project, including all its phases and processes involved to achieve them.

3. **Project Architecture**
   Draw the architecture indicating all the different components that makes up this system and write a brief description of the architecture. An example of an architecture is shown below.

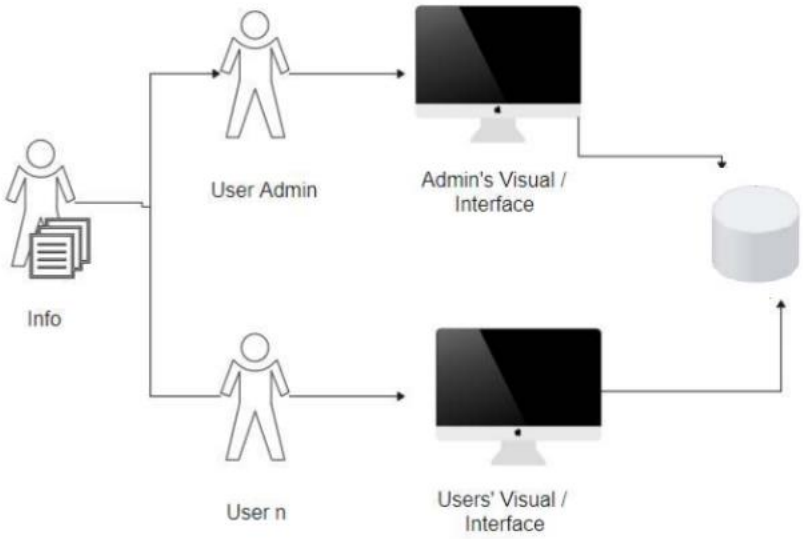


Figure 1 System architecture

4. **UML Class Diagram**
   Generate class diagram that shows an overview of how all current classes and packages work together for your project. An example of a class diagram is shown below.

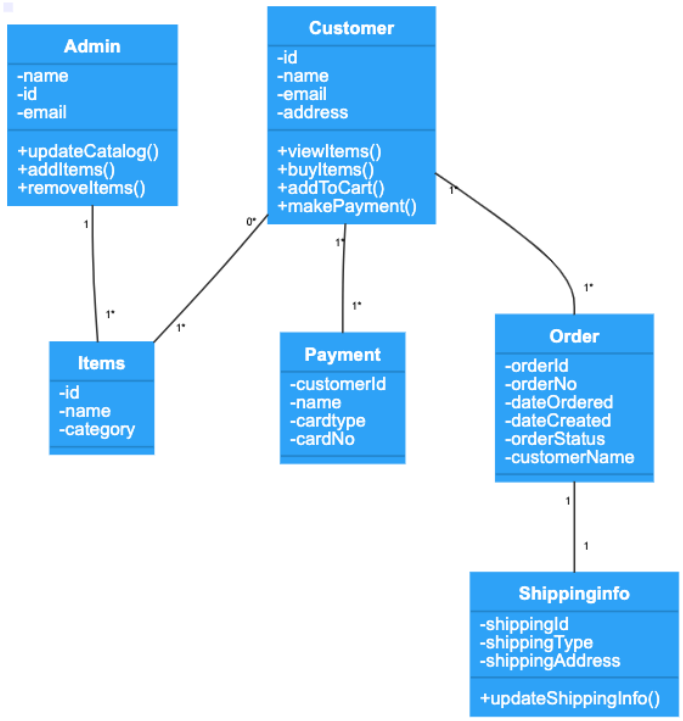


Figure 2 UML class diagram

5. **OOP Concepts Used**
   Talk about all the different kinds of oop concepts used and describe them each in details. Also describe how you have used them to assemble your project.

6. **Code Description**
   Identify 3 key features of your system and present the sample code together with short descriptions to describe how they are used to implement specific parts of your system.

7. **System Testing**
   Provide screenshots of tests run on your completed system.

8. **Group members**
   Explain the individual responsibility of each team member inside the project.

9. **Conclusion and future works**
   It is concluded that the main objective of the application is to help computer science students understand the basics of Java. By browsing through the application and looking at the code for each graphical interpretation, students should be able to easily understand the implementation. Future work could involve adding more categories which are more detailed and have additional items. This part of the documentation is reserved for conclusion and future possible improvements on the project itself

Project Introduction:
The main idea behind the project Cubicle is to create a terminal based interface that allows anyone who is looking for a property/building to live or purchase (House, Apartment, etc) to collect tons of different properties that they might want and then use our features to make the search easier and more organized.

Project Description:
The goal of the project is to create a community-based project for a growing pain which is searching for places to live in after graduation or because of growing property prices. With our filtering features, you can create sub lists of your overall list of properties you want and save that to other files.

The Objective of the Project

1. Reading property data from a file and storing it in an ArrayList of properties in a PropertySystem class that oversees everything within the main function regarding a property. If they don't have a file they can get a clear list to start off with.
2. Allow users to create properties to save to a list and delete properties that they don't want to include anymore.
3. Utilize a filtering system to allow users to narrow down their searches and have optional saving in a new file for that sub list.
4. Create a beautiful graphical design with text-based drawings and subsections to distinguish between actions
5. Ensure proper input validation and error handling.

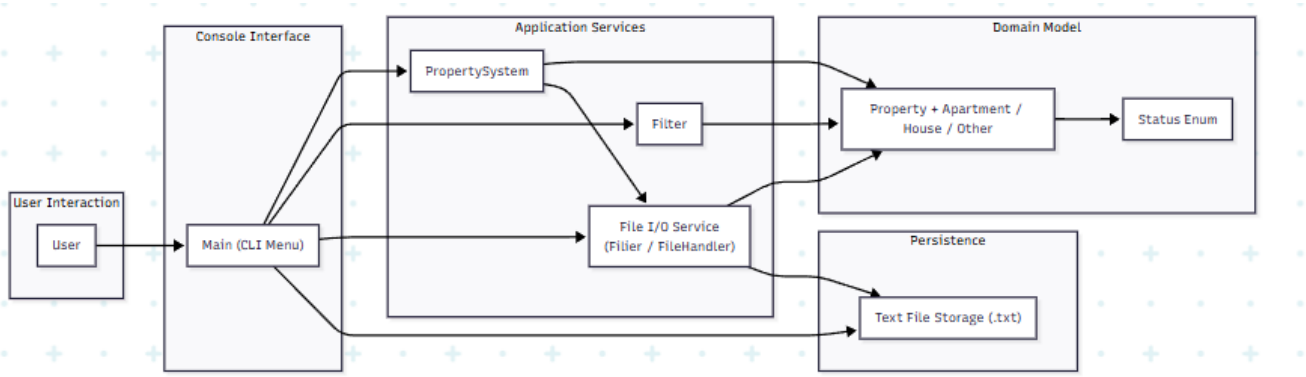
Project Phases

1. **Input Handling** – Menu System, validation, and user interaction.
2. **File Handling** – Reading and writing property data in a format that works each time the program reads a file made by it
3. **Data Storage** – Managing properties using dynamic structures (An arraylist of the Property

class)

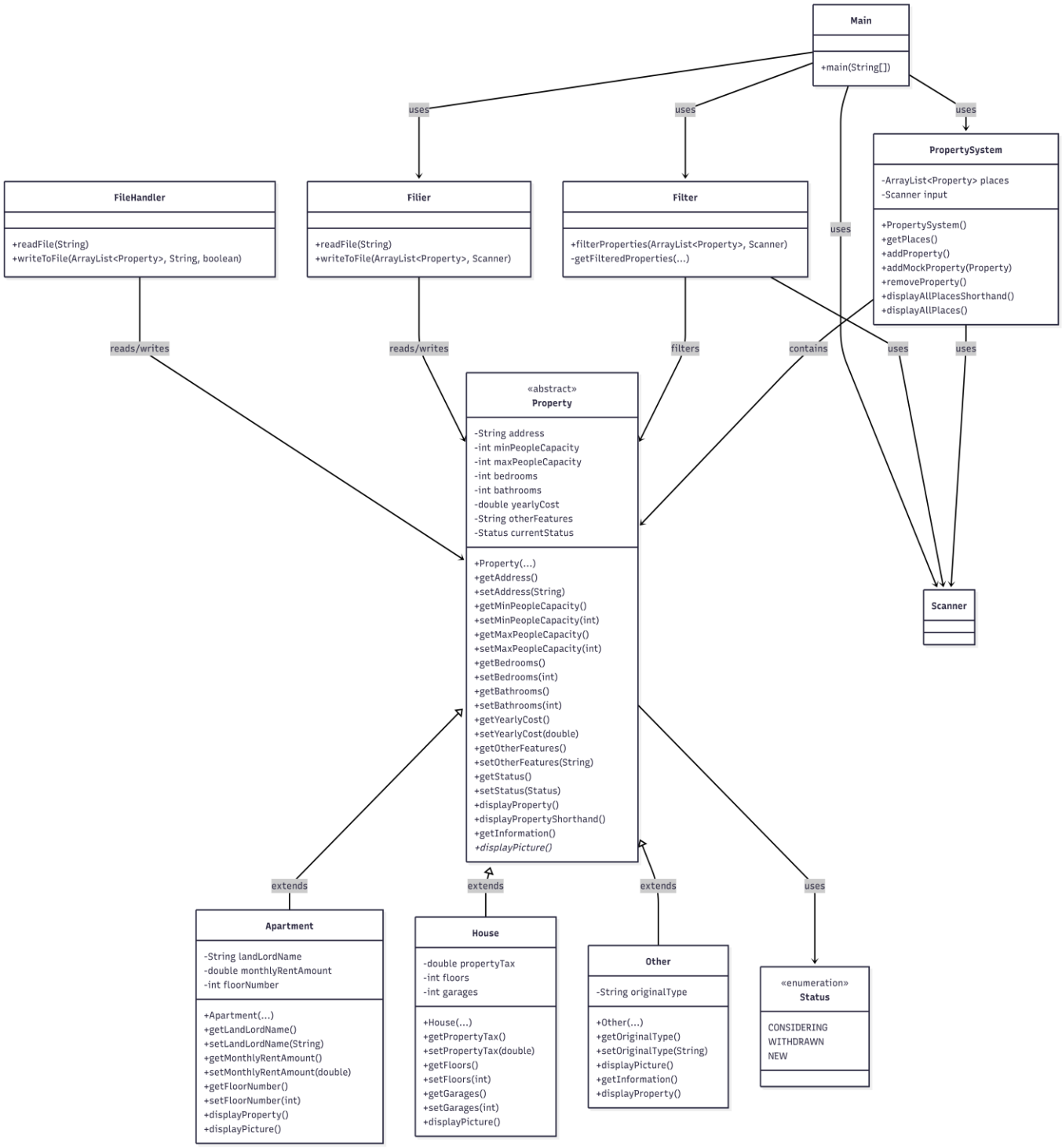
4. **Filtering and Search** – Users can decide the min and max values for a multitude of values based on their preferences like bathrooms, bedrooms, etc.
5. **Display System –** Ensure that everything is presented in a clear format that doesn't strain the eyes or make the viewers question what they are seeing.

Project Architecture



The architecture for Cubicle is as follows. The user interface is where the user can give input to the questions on the terminal like if they wanted to override the file, if they wanted to give the yearly cost of the property, or if they wanted to filter based on Status. The application services handles the core logic of the system. The Main class serves as the entry point and directs user actions to the PropertySystem which can then apply filtering through the Filter class or call File Input and Output through the Filier class. The domain model consists of the abstract Property class and it's subclasses of Apartment, House, and Other which all use a Status enum. Lastly, the persistence layer is there for storing and retrieving data, specifically using the Filier's static methods of readFromFile and writeToFile which saves it to a .txt file.

UML Class Design



OOP Concepts Used

- **Encapsulation:**

+ Classes like Property, House, and Apartment use private access modifiers for their internal variables (e.g., price, address, number of bedrooms). The PropertySystem and Main classes cannot modify these variables directly so they must interact through public getter and setter methods. This protects the data integrity of the system, allowing us to enforce validation (such as preventing a negative property price) before the state changes.

- **Inheritance:**

+ A base Property class that defines the core attributes shared by all living spaces, such as address, square footage, and budget. The House and Apartment classes extend this base class so they automatically inherit the shared attributes and can also define their own unique fields (e.g., a House might have a yard size, while an Apartment might have a floor number or specific amenities), reducing code duplication.

- **Polymorphism:**

+ The PropertySystem stores all properties in a single ArrayList<Property>. We can safely add both House and Apartment objects to this list. When the user searches for a living space and the system iterates through the list to call a method like displayInfo() or toString(), Java dynamically invokes the correct, overridden version of that method for either a House or an Apartment.

- **Abstraction:**

+ The main application menu does not need to understand the underlying complexities of BufferedReader, FileWriter, or the pipe-delimited (|) serialization format. It simply calls high-level methods like saveData() or loadData(). Furthermore, treating Property as an abstract concept ensures that a developer cannot mistakenly instantiate a generic property; they must create a concrete House or Apartment.

Code Description

Code Segment 1:

```java
public static void writeToFile(ArrayList<Property> list, Scanner input){

    if(list.isEmpty()){
        System.out.printf(format: "There is no property to write to a file :(");
        return;
    }

    //ask user which file to write to and if they want it to overwrite their existing work
    boolean append;
    String appendChoice;
    System.out.print(s: "Enter a file you wish to write the information into: (don't include .txt): ");
    String fileName = input.nextLine().concat(str: ".txt");
    System.out.print(s: "Do you wish to overwrite the file? (Y/N): ");
    appendChoice = input.nextLine();
    append = !appendChoice.equalsIgnoreCase(anotherString: "y");

    try (BufferedWriter writer = new BufferedWriter(new FileWriter(fileName,append))) {
        for(Property property : list){
            //getInformation has the properties seperated by | for easier reading/parsing for readFileMethod
            writer.write(property.getInformation());
            writer.newLine();
        }
        System.out.printf(format: "Successfully written to %s\n", fileName);
    } catch (Exception e) {
        System.out.println(x: "There was an error writting to the file please try again");
    }
}
```

This is one of most important code segments in that it ensures that the data that the user gives (from reading a file, adding new properties, and deleting properties) persists after the user closes the program. Basically what this function does is that if there is nothing in the list it won't even try to write into a file, but if there is a property in the list, it will ask the user to give a file and if they want it overwritten. After the file is written into, the user is given a message confirming that a file is populated with the data.

Code Segment 2:

```
ArrayList<Property> validProperties  = getFilteredProperties(list, minBedrooms, maxBedrooms, minBathrooms,
maxBathrooms, minCapacity, maxCapacity, minFloors, maxFloors, minYearCost, maxYearCost, minTax, maxTax, inStatus);

if(validProperties.isEmpty()){
    System.out.println(x: "There were no properties that matched your criteria :(");
}
//display the filtered properties
else{
    System.out.println(x: "---FILTERED PROPERTIES---");
    for(int i = 0; i < validProperties.size(); i++) {
    System.out.println(x: "---------------------------");
    System.out.println("Filtered Property " + (i+1));
    validProperties.get(i).displayProperty();
    System.out.println(x: "---------------------------");
    input.nextLine();
    }
    //ask user if they want to save the filtered properties to a file
    String saveToFile;
    System.out.print(s: "Would you like to save the filtered properties to a file? (Y/N): ");
    saveToFile = input.nextLine();
    if(saveToFile.equalsIgnoreCase(anotherString: "y")){
        Filier.writeToFile(list, input);
    }
}
```

This code segment deals with the filtering aspect of our program. What is happening here is that the getFileteredProperties function takes all those parameters and gives a filtered list of properties that match the criteria. If there are some properties in that subset, then it will display the properties onto the screen, and after that, it will prompt the user if they want to save it to a file (ie filteredData.txt).

Code Segment 3:

```
//allow user to read from file at start or go manual
System.out.print(s: "Load properties from a saved file? (Y/N): ");
if (input.nextLine().trim().equalsIgnoreCase(anotherString: "y")) {
    System.out.print(s: "Enter the file name to load (don't include .txt): ");
    fileName = input.nextLine().trim().concat(str: ".txt");
    for (Property p : Filier.readFile(fileName)) {
        system.addMockProperty(p);
    }
    if(!system.getPlaces().isEmpty()){
        System.out.println("Properties loaded from " + fileName);
    }
}
```

The last code segment occurs at the start of our program when the Cubicle title is displayed. Basically what it does is that it allows a user to give the program a file to pre-populate the arrayList in the PropertySystem object with properties (this usually occurs after a user already used the program and

written the information into a file or if the user manually written the system we used for parsing the data). This fulfills the File Input functionality of our program and gives a user a way to quickly bring in their old properties instead of rewriting all of it again.

System Testing

```
---WELCOME TO CUBICLE---
Load properties from a saved file? (Y/N): Y
Enter the file name to load (don't include .txt): SUPERBOB
Properties loaded from SUPERBOB.txt

---CUBICLE MENU---
1. Display All saved Properties
2. Display All saved Properties Shorthand
3. Add a property to the list
4. Delete a saved property
5. Filter Properties based on criteria
6. Write saved properties to a file
7. Exit the Program (Make sure to save your information!)
-----------------
Enter your choice: 2

A simplified display of the properties:
---------------------------
PROPERTY #1: $500000.00 | CONSIDERING | 1990 Blue Lake Blvd Arlington TX, 76040
PROPERTY #2: $20000.00 | NEW | 1241 Super Lake Ave
PROPERTY #3: $750000.00 | WITHDRAWN | 88 Cedar Ridge Dr
---------------------------

---CUBICLE MENU---
1. Display All saved Properties
2. Display All saved Properties Shorthand
3. Add a property to the list
4. Delete a saved property
5. Filter Properties based on criteria
6. Write saved properties to a file
7. Exit the Program (Make sure to save your information!)
-----------------
Enter your choice: 7
Thank you for using Cubicle!
```

```
---WELCOME TO CUBICLE---
Load properties from a saved file? (Y/N): Y
Enter the file name to load (don't include .txt): BOB
There was an error accessing that file in your folder. We will default to manually adding

---CUBICLE MENU---
1. Display All saved Properties
2. Display All saved Properties Shorthand
3. Add a property to the list
4. Delete a saved property
5. Filter Properties based on criteria
6. Write saved properties to a file
7. Exit the Program (Make sure to save your information!)
-----------------
Enter your choice: 1
There are no saved properties!, Please add some if you wish to view your properties

---CUBICLE MENU---
1. Display All saved Properties
2. Display All saved Properties Shorthand
3. Add a property to the list
4. Delete a saved property
5. Filter Properties based on criteria
6. Write saved properties to a file
7. Exit the Program (Make sure to save your information!)
-----------------
Enter your choice: 3

---GENERAL INFORMATION OF PROPERTY---
Enter the type of the property (Ex House / Apartment / Studio): HOuse
Status of the property (CONSIDERING, NEW, WITHDRAWN): NEW
Address: 1444 Physics Drive
Min and Max people capacity (seperated with a space): 1 3
Number of bedrooms and bathrooms (seperated with a space): 2 2
Other features of the property: It has a pool and a telescope

---SPECIFIC INFORMATION ABOUT PROPERTY---
Yearly Cost: 300000
Property Tax %: 1.5
Floors: 2
Garages: 2
Succesfully added the property!

---CUBICLE MENU---
1. Display All saved Properties
2. Display All saved Properties Shorthand
3. Add a property to the list
4. Delete a saved property
5. Filter Properties based on criteria
6. Write saved properties to a file
7. Exit the Program (Make sure to save your information!)
-----------------
Enter your choice: 7
Thank you for using Cubicle!
```

```
---WELCOME TO CUBICLE---
Load properties from a saved file? (Y/N): Y
Enter the file name to load (don't include .txt): SUPERBOB
Properties loaded from SUPERBOB.txt

---CUBICLE MENU---
1. Display All saved Properties
2. Display All saved Properties Shorthand
3. Add a property to the list
4. Delete a saved property
5. Filter Properties based on criteria
6. Write saved properties to a file
7. Exit the Program (Make sure to save your information!)
-----------------
Enter your choice: 1

After a property is displayed press enter to load the next
---------------------------
PROPERTY #1
    ________________
   /                \
  /__________________\
   |  __    __    __ |
   | |__|  |__|  |__| |
   |                  |
   |     ________     |
   |    |   __   |    |
   |    |  |  |  |    |
   |___ |__|__|__|____|
       |          |
       |__________|

2 FLOOR HOUSE
Current Status: CONSIDERING
For 2-6 People
Address: 1990 Blue Lake Blvd Arlington TX, 76040
Bedrooms: 4 | Bathrooms: 3
Yearly Cost: $500000.00
Property Tax: 1.20%
Garages: 2
Other Information: Pool, Solar Panels
---------------------------

---------------------------
PROPERTY #2
    ___________
   |   _    _   |
   |  |_| |_|   |
   |   _    _   |
   |  |_| |_|   |
   |   _    _   |
   |  |_| |_|   |
   |___________|
       |  |
       |__|

APARTMENT ON FLOOR 3
Current Status: NEW
For 1-4 People
Address: 1241 Super Lake Ave
Bedrooms: 2 | Bathrooms: 2
Yearly Cost: $20000.00
Monthly Rent: 1500.0
```

Group Members

Andy Ta: Created the GitHub repository and set up the basic classes and structure of the project. I created the abstract Property class, the 3 subclasses that inherited it (House, Apartment, Other), the PropertySystem class, and the output system that the user saw in Main(). I also worked on the Filier static method writeToFile and brought all the functionality from Duc's and Andrew's code to work coherently with the classes I wrote and with each's others code. Lastly, I wrote documentation for Project Introduction, Project Description, Code Description, System Testing, and the conclusion section.

Duc Minh Nguyen: Implemented the file I/O system to serialize the dynamic `ArrayList` of properties, allowing the application to permanently save and load state between sessions using a robust, custom pipe-delimited text format designed to prevent parsing collisions with complex street addresses, wrote documentation for Project Architecture, OOP Concepts Used, UML Diagram, Future Works

Andrew Butts: Designed and coded filtering functionality to work with the given classes and their associated attributes. Specifically: made filterProperties and getFilteredProperties methods along with most of the code in Filter.java to allow them to function.

Conclusion and Future Works

- Overall, this project wasn't so bad since it was an idea that I (Andy) had since 12th grade of high school for his Java class. Our main goal was to make something that helps people collect and organize all the places they found online that they might like to purchase in the near future, and after finishing the program, I felt like we did just that. We demonstrated good usage of object-oriented programming principles like abstraction, polymorphism, and encapsulation together with file handling and data structures like ArrayList to build a functional application.

- Future possible improvements for the project could be:

+ Database Migration: As the project grows, it needs a better database. We can upgrade the current storage system to a robust relational database management system which would allow for much faster, concurrent querying of property data.
+ Advanced Search Algorithm: Maybe we can implement more sophisticated data structures to optimize the filtering speed for users searching with multiple simultaneous constraints.
+ Graphical User Interface Integration: Improve accessibility and allow for visual representations of properties.
+ Add a scoring system where we can rate each place based on a scale of 0-100 based on how many criteria points a property fulfills.